

A thousand and one

The wind of change is blowing through the market and Acorn's Product Development chief, Peter Bender, has made no secret of the fact that the company is examining Taos as a possible operating system of the future. But what is it? And why is Acorn interested?

The Murb says that Taos is a multi-tasking, multi-threaded, multi-processor, parallel processing system – it's all things to all users and probably makes tea as well – but that doesn't really help. Let's take these terms one at a time and see where they take us.

Jargon explained

Multi-tasking is the easy one; our machines have been doing this trick for seven years. It's simply the ability to run more than one program at a time, running the alarm clock and the CD player software together, for example.

Multi-threaded means using the same code to do more than one task at a time. With the standard Acorn operating system, multi-threading usually only applies to a single program; if a program can handle more than one document at a time, it's multi-threaded.

However, it can get more complex – you might get a piece of code that is executing which passes control to another routine, at which point the same code can get called again from another place and get executed. At this point you would have one piece

of code with two 'threads' running through it.

Multi-threading is not very common with RISC OS, though it does happen in the background a reasonable amount. Some routines can do it, some can't. With Taos, every routine can.

Then there's multi-processor; we might be able to plug a 486 PC card into the Risc PC and run Windows, but that doesn't make it multi-processor – at least not in this sense. What's intended here is to show that Taos will run on any processor. Taos programs don't require a huge effort to convert from platform to platform either – any processor for which there is a Taos kernel and interpreter can run any Taos program, without modification.

But it's even more useful than that: if you have more than one processor – and the kernel knows about the other processors – Taos automatically distributes the executing code across the available processors and thus runs as a parallel processing system. And with parallel

processing every additional processor adds power.

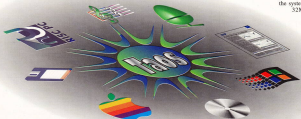
There does come a point where the sending and receiving of messages between processors can cancel out any increase in power that adding another processor brings. However, it takes quite a few processors to do that and, theoretically, Taos can handle over two million of them.

Taos for Acorn

Just one week before I was due to hand in this review I received my advance copy of the Taos development system for RISC OS. It's not entirely finished and won't be released for a while yet, but further changes are really a matter of bells and whistles rather than any fundamental alterations.

Taos comes in two varieties: TUI and GUI. The first is the *textual user interface* and the other is the *graphical user interface*; each is a separate application. There's the ARM6 kernel application which provides the Taos environment and needs, by default, 6Mb of RAM to run in,

though it can run in less if needed. The final part of the system is the 52Mb DOS



Steve Turnbull opens the Pandora's box that is the Taos operating system.

tools

partition, 32Mb is not an absolute requirement – the current size of the installed Taos code is just 8Mb – but it allows for future expansion. As Taos was originally developed on a PC it expects filenames to be in the old 8+3 DOS format and it needs virtually unlimited files in a directory. The Acorn filing system can't handle 11 character filenames or more than 37 objects in a directory (yet). Unfortunately, Taos must have an identical file system on every machine it runs on; hence the DOS partition.

One of the delays in releasing the system was the need to produce an installation routine that can create the partition without the need for the PC emulator or PCcopy application, which is what I was able to use. From there, installation is just a matter of using a ver-

sion of *Spark* to unpack the .zip files off the floppies and into the partition. Once this has been done, Taos will run.

It's tool time

When any Taos program – or Taos itself – is run for the first time there is an enormous amount of disc activity and it can take quite a time to get up and running. One of the unique features of Taos is its use of tools: it's almost true to say that where most programs have subroutine calls, Taos has accesses to another tool.

Each tool is a very short (100-150 byte) piece of code that carries out a very precise and quick action, though it might require further calls to other tools to achieve it. It is located on disc and is accessed via its file name and path. The good news is that once it's loaded, Taos

will hold it in memory until it's no longer needed and its space is needed by another tool. Bearing in mind that each tool is very short, you can get a lot of them into the space provided.

For example let's say you've written a piece of code that needs to calculate the SIN of an angle. To do this your code would call the MATH/SIN tool or, if a floating point version were needed, MATH/SINF. Both tools are stored in the MATH directory in the partition. Now, the MATH directory contains all the tools to implement the C math library, but the nature of Taos means they are all separate and any program can call them, regardless of the language it's being written in. No special interface is required – try that on any other machine!

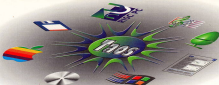
Another reason for having short lengths



Francis Chang, Chairman of Taos systems.



Chris Hindey, Technical Director and founder of Taos.



of code is the way Taos operates as far as different processors are concerned. Until now I've rather glibly stated that code is translated as it's loaded. Let's take a closer look at that.

Taos programs are generally written in VP code – VP standing for Virtual Processor. VP code is an idealised set of instructions and directives that, while not elegant, is certainly compact. When a call is made to a tool that isn't loaded, Taos checks the path to the directory and examines what's there. Routines that end in .00 are VP code routines, so MATHSIN.00 is the VP code for calculating the SIN of a number.

If this were the only one available, Taos would pull the code in and translate it into ARM code – or 486 code, or Pentium, or 68000, or whatever – and then execute it. Any further calls to the code would not require the translation step as long as it's in memory. If it got purged it would have to be reloaded and re-translated.

But the load check doesn't stop at VP code. It may be that a particular function can be re-coded especially efficiently for a specific processor, so it's possible to

write a machine-specific routine, give it the same name, but make the file extension a different number relating to that processor. The number for the ARM209 series is .05, and .06 for ARM67 series. This means that when Taos is looking for the code to execute, it checks for routines that are specific to the processor first and for VP code last.

What's in it for Acorn?

While Acorn may not currently be the biggest manufacturer of computers in the world, it's certainly one of the most innovative, and within the next two years Acorn is very likely to produce a 3D user-interface for its machines, and for responsive 3D work you need power.

Not only is Sinter developing the five-ARM system for existing Risc PCs, but Acorn has also revealed that it's planning to produce a multi-processor machine which has exactly the kind of power to deliver an effective and responsive 3D system. What Acorn doesn't have is an operating system.

Up to now, all parallel processing computers have required finely-tuned

operating systems that usually have some sort of hierarchical control methodology. Taos throws this out of the window – there is no boss processor, each one operates on its own and uses an algorithm to compare its own 'power' with those of its neighbours. If one of its neighbours has more power it gives it the process to execute, then gets on with its own work.

Taos absorbs more power in managing than the clever OSs, but it makes up for it with simplicity and robustness. This is precisely the philosophy that Acorn uses in the development of its hardware and software: make it clean and simple and it'll work fast.

The Taos GUI, in my opinion, is horrible. It resembles Windows and lacks any subtlety. However, this is not important – Taos can build any GUI you want, whether it be the 3D-Rooms of the future, or the RISC OS 3 of the present. What's more important is that once it works it can be put on any processor that runs Taos, and suddenly you'll have Acorn's Rooms or RISC OS on any machine you care to mention.

The future may well be Taos.

ATU

A brief history of Taos

Taos was not initially created as a multi-processor operating system. Its first incarnations (in 1988) were as a macro language to help port games between different platforms. But its inventor Chris Hinley decided to target the forthcoming 32-bit machines rather than aiming at the popular machines available then.

During the early part of the 1990s, development by a core team of enthusiasts continued until Hands Charlie joined the group, setting up Taos Systems with Chris and Tim Moore (who had also been a major part of the development team) and organising financing from Japan.

Since then, development has continued apace, the team has grown, the company has managed to sustain itself without creating any debt whatsoever, and has now established a group of major consumer electronics companies as customers which assures it of long-term stability. Some of these companies will be using Taos as their primary operating system.

As a product, Taos is able to offer an unusual selection because its features allow manufacturers to utilise a single operating system that can be used with PDAs (personal digital assistants), mobile phones, faxes, printers, VCRs, set-top boxes, personal computers, games machines, virtual reality, video servers and so forth, regardless of the processor and using common software. Within the games market, there will be a developer's kit launched which allows games to be written once which will then run across dissimilar workstations and games consoles.

Taos Systems considers its competitors to be such things as OS-5, pSOS, WinOS, UNIX and the IBM Microkernel, but these are mostly processor-specific and lack the generalisation of Taos.

Taos is portable, exactly the same code runs on any processor or machine for which a Taos translator exists with minimal speed degradation – given that there has to be some slowdown since code translation is involved. It's also highly compact – the ARM kernel occupies just 7K while the Intel one occupies 10K.

Taos is also a parallel operating system in which it self-load balances using an optimising algorithm across the network, capable of distributing across up to 2,000,000 processors in a single network. So long as the bandwidth between processors is sufficient, the performance on the network will increase. This is the result of using a masterless network rather than traditional techniques.

By being parallel and portable, Taos can run with dissimilar microprocessors across the same network, with everything being shared, even though the processors are dissimilar. Thus, an ARM-based workstation can take advantage of Intel 486s as they are added to the network and will distribute the process load according to the power available on each processor. This provides consumers with future-proofing as they are able to extend networks based on the latest technologies as they become available, not just as historical decisions. As interest in Taos increased there was pressure to permit outside development of actual software. However, this was resisted to allow the OS for Taos to be put together into a usable structure, which now exists.

A number of global consumer electronics companies involved with Taos will be announcing their support formally over the summer. There is also going to be more public support for Taos from the cable and satellite companies in the months ahead as they look for an effective solution for interactive television.